

xv6 启动文件系统追踪

从一次启动的 fs-trace 日志重建出每个文件系统的 superblock 和目录项，以及每次路径查找怎样从 inode 走到目录项、再走到数据块。图里的数值都来自日志；日志没有打印、由上下文推出的值标为“推断”。

日志行数 4409

BSIZE 1024 B

dirent 32 B (DIRSIZ 30)

dinode 64 B, 16 个/块

superblock 7

挂载 6

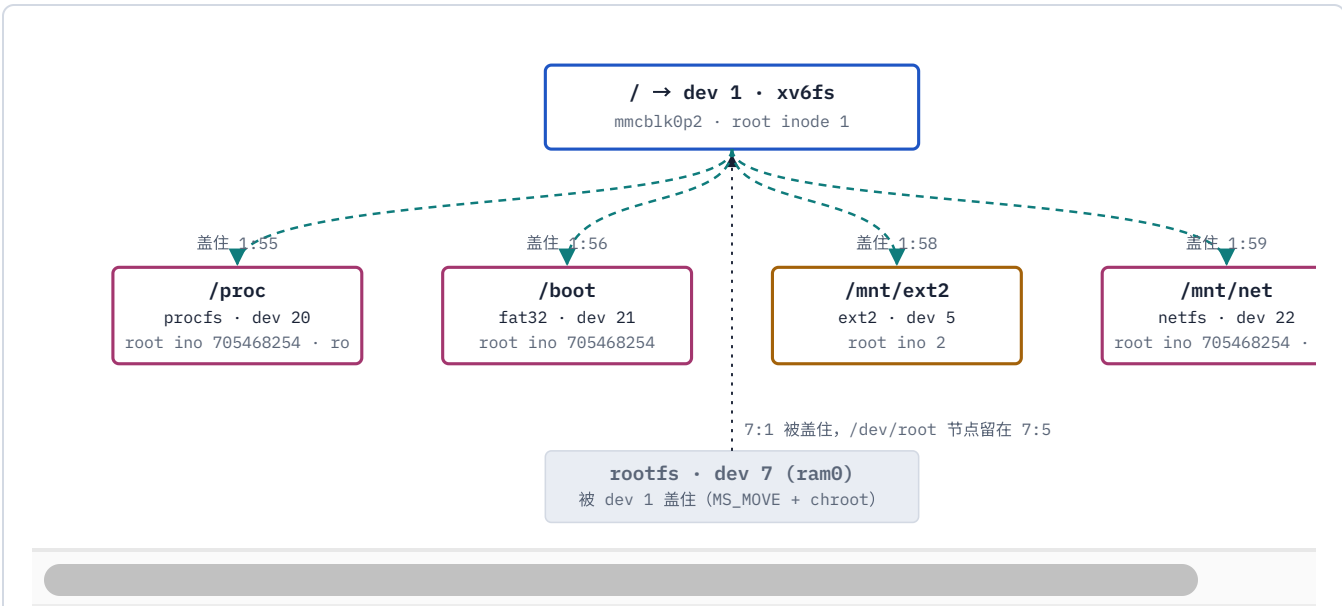
inode 41

路径查找 37

[挂载与命名空间](#) [各文件系统](#) [路径查找图](#) [从日志看到的问题](#)

挂载与命名空间

启动结束时，/ 是 dev 1（mmcblk0p2 上的 xv6fs），rootfs（dev 7）被压在下面。其余五个文件系统各盖住根文件系统里的一个空目录；虚线箭头上的 1:54 这类编号是被盖住的目录 inode。



挂载顺序（日志行号）：/root ← xv6fs dev 1（第 2227 行）；/root/dev ← xv6fs dev 8（第 2355 行）；/proc ← procfs dev 20（第 3729 行）；/boot ← fat32 dev 21（第 3779 行）；/mnt/ext2 ← ext2 dev 5（第 3856 行）；/mnt/net ← netfs dev 22（第 3938 行）。前两项是 prepare_namespace() 里的 /root 和 /root/dev，MS_MOVE 后分别成为 / 和 /dev。

各文件系统：superblock、目录项与 inode

xv6fs 有磁盘 superblock（fill_super 读出的块 1）和内存里的 VFS super_block；ext2、

procfs、fat32、netfs 在日志里只有 VFS super_block。块区间按 superblock 字段计算：inode 区 = ninodes/16+1 块，位图 = size/8192+1 块，其后是数据区。

dev 7 **rootfs · xv6fs** 来源 ram0

prepare_namespace() 第 1 步用 ramfs_mkfs() 在 32 块的内存盘里造出的临时根。日志里能看到它建 /dev、/dev/console、/root 和后来的 /dev/root。

boot

super

inode

bitmap

0

1

2-3

4

5-31

块区间不按比例绘制。数据区第一块 = 5，与根目录 inode 的 addrs[0] 对得上。

inode 区 (每块 16 个 dinode, 每个 64 B)

数据区 (从块 5 起)

inode 块 2
ino 0-15 (ino 0 不用)

#1	ino 1	/	T_DIR
#2	ino 2	/dev	T_DIR
#3	ino 3	/dev/console	T_DEVICE
#4	ino 4	/root	T_DIR
#5	ino 5	/dev/root	T_DEVICE

块 5 · 目录 /
ino 1 的 addrs[0] · 4 个目录项 × 32 B

off 0	.
off 32	..
off 64	dev
off 96	root

块 6 · 目录 /dev
ino 2 的 addrs[0] · 4 个目录项 × 32 B

off 0	.
off 32	..
off 64	console
off 96	root

块 7 · 目录 /root
ino 4 的 addrs[0] · 2 个目录项 × 32 B

off 0	.
off 32	..

黑色箭头：inode 的 addrs[] 指向数据块；蓝色箭头：目录项里的 inode 号指回 inode 区的那个 dinode。绿色底的目录项是启动时新建的。只画出日志里读到过的 inode 和块。

磁盘 SUPERBLOCK (块 1)

VFS SUPER_BLOCK (REGISTER_SUPER)

magic	10203040	dev	7
size	32	type	xv6fs
nblocks	27	rootino	1
ninodes	16	readonly	0
nlog	0	logged	1
logstart	2	pathbased	0
inodestart	2	iop	0xffffffff80000b6b10
bmapstart	4	priv	0x0000000000000000

目录项 (DENTRY)

/ ino 1

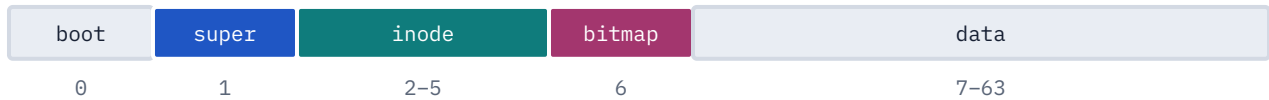
```

/ 110 1
dev/ → ino 2 · off 64 · T_DIR 启动时创建
  console → ino 3 · off 64 · T_DEVICE 启动时创建
  root → ino 5 · off 96 · T_DEVICE 启动时创建
root/ → ino 4 · off 96 · T_DIR 启动时创建

```

dev 8 **devtmpfs · xv6fs** 来源 ram1

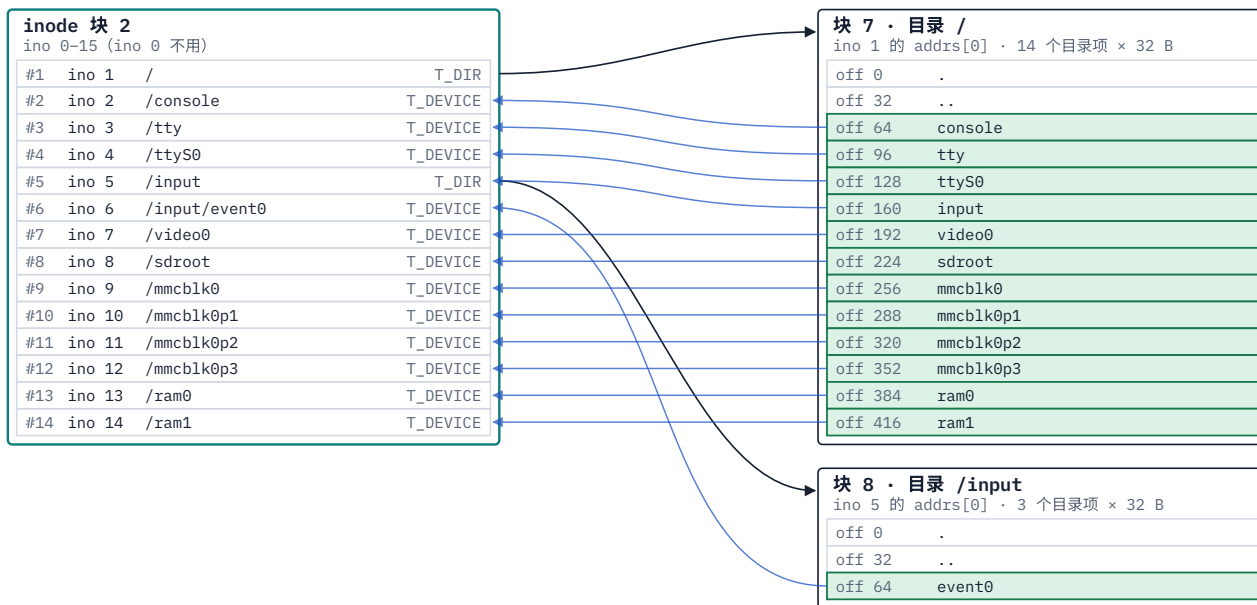
devtmpfs_init() 在 64 块的 ram1 上建出 12 个设备节点；它先挂在 /root/dev（盖住 1:54），MS_MOVE 后路径变成 /dev。



块区间不按比例绘制。数据区第一块 = 7，与根目录 inode 的 `addrs[0]` 对得上。

inode 区 (每块 16 个 dinode, 每个 64 B)

数据区 (从块 7 起)



黑色箭头：inode 的 `addrs[]` 指向数据块；蓝色箭头：目录项里的 inode 号指回 inode 区的那个 dinode。绿色底的目录项是启动时新建的。只画出日志里读到过的 inode 和块。

磁盘 SUPERBLOCK (块 1)

magic	10203040
size	64
nblocks	57
ninodes	48
nlog	0
logstart	2

VFS SUPER_BLOCK (REGISTER_SUPER)

dev	8
type	xv6fs
rootino	1
readonly	0
logged	1
pathbased	0

inodestart	2	iop	0xffffffff80000b6b10
bmapstart	6	priv	0x0000000000000000

目录项 (DENTRY)

/ ino 1

console → ino 2 · off 64 · T_DEVICE 启动时创建

tty → ino 3 · off 96 · T_DEVICE 启动时创建

ttyS0 → ino 4 · off 128 · T_DEVICE 启动时创建

input/ → ino 5 · off 160 · T_DIR 启动时创建

event0 → ino 6 · off 64 · T_DEVICE 启动时创建

video0 → ino 7 · off 192 · T_DEVICE 启动时创建

sdroot → ino 8 · off 224 · T_DEVICE 启动时创建

mmcblk0 → ino 9 · off 256 · T_DEVICE 启动时创建

mmcblk0p1 → ino 10 · off 288 · T_DEVICE 启动时创建

mmcblk0p2 → ino 11 · off 320 · T_DEVICE 启动时创建

mmcblk0p3 → ino 12 · off 352 · T_DEVICE 启动时创建

ram0 → ino 13 · off 384 · T_DEVICE 启动时创建

ram1 → ino 14 · off 416 · T_DEVICE 启动时创建

dev 1 根文件系统 · xv6fs 来源 mmcblk0p2

mmcblk0p2 上真正的根文件系统，带 30 块日志。fill_super 读出的 superblock 在第 2182 行。

boot	super	log	inode	bitmap	data
0	1	2-31	32-44	45-49	50-32767

块区间不按比例绘制。数据区第一块 = 50，与根目录 inode 的 `addrs[0]` 对得上。

inode 区 (每块 16 个 dinode, 每个 64 B)

inode 块 32			
ino 0-15 (ino 0 不用)			
#1	ino 1	/	T_DIR
#2	ino 2	/bin	T_DIR
⋮			
#7	ino 7	/bin/init, /init	T_FILE
⋮			
#10	ino 10	/bin/login	T_FILE

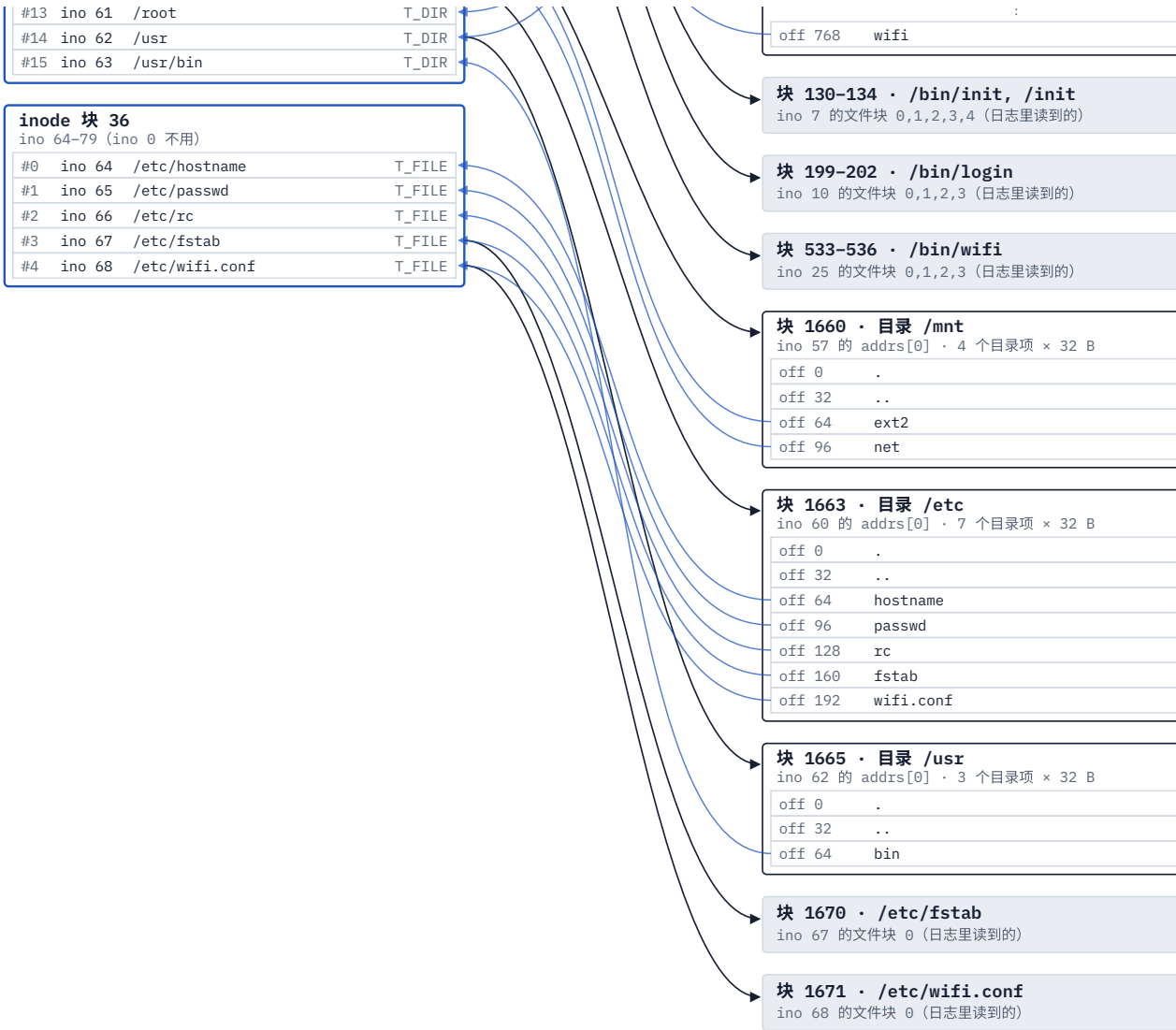
inode 块 33			
ino 16-31 (ino 0 不用)			
#9	ino 25	/bin/wifi	T_FILE

inode 块 35			
ino 48-63 (ino 0 不用)			
#6	ino 54	/dev	T_DIR
#7	ino 55	/proc	T_DIR
#8	ino 56	/boot	T_DIR
#9	ino 57	/mnt	T_DIR
#10	ino 58	/mnt/ext2	T_DIR
#11	ino 59	/mnt/net	T_DIR
#12	ino 60	/etc	T_DIR

数据区 (从块 50 起)

块 50 · 目录 /	
ino 1 的 <code>addrs[0]</code> · 11 个目录项 × 32 B	
off 0	.
off 32	..
off 64	bin
off 96	init
off 128	dev
off 160	proc
off 192	boot
off 224	mnt
off 256	etc
off 288	root
off 320	usr

块 51 · 目录 /bin	
ino 2 的 <code>addrs[0]</code> · 25 个目录项 × 32 B	
off 0	.
off 32	..
⋮	
off 192	init
⋮	
off 288	login



黑色箭头：inode 的 addr[] 指向数据块；蓝色箭头：目录项里的 inode 号指回 inode 区的那个 dinode。绿色底的目录项是启动时新建的。只画出日志里读到过的 inode 和块。

磁盘 SUPERBLOCK (块 1)		VFS SUPER_BLOCK (REGISTER_SUPER)	
magic	10203040	dev	1
size	32768	type	xv6fs
nblocks	32718	rootino	1
ninodes	200	readonly	0
nlog	30	logged	1
logstart	2	pathbased	0
inodestart	32	iop	0xffffffff80000b6b10
bmapstart	45	priv	0x0000000000000000

目录项 (DENTRY)

/ ino 1

► bin/ → ino 2 · off 64 · T_DIR

```
init → ino 7 · off 96 · T_FILE
dev/ → ino 54 · off 128 · T_DIR
proc/ → ino 55 · off 160 · T_DIR → 挂载 procfs dev 20
boot/ → ino 56 · off 192 · T_DIR → 挂载 fat32 dev 21
mnt/ → ino 57 · off 224 · T_DIR
|   ext2/ → ino 58 · off 64 · T_DIR → 挂载 ext2 dev 5
|   net/ → ino 59 · off 96 · T_DIR → 挂载 netfs dev 22
etc/ → ino 60 · off 256 · T_DIR
|   hostname → ino 64 · off 64 · T_FILE
|   passwd → ino 65 · off 96 · T_FILE
|   rc → ino 66 · off 128 · T_FILE
|   fstab → ino 67 · off 160 · T_FILE
|   wifi.conf → ino 68 · off 192 · T_FILE
root/ → ino 61 · off 288 · T_DIR
usr/ → ino 62 · off 320 · T_DIR
|   bin/ → ino 63 · off 64 · T_DIR
```

dev 5 **ext2 · ext2** 来源 mmcblk0p3

ext2 的磁盘 superblock 由 ext2init() 读，不经过 fill_super 跟踪点，日志里只有 VFS 这一层。根 inode 号是 ext2 的约定值 2。

磁盘 SUPERBLOCK (块 1)
日志里没有磁盘 superblock。

VFS SUPER_BLOCK (REGISTER_SUPER)	
dev	5
type	ext2
rootino	2
readonly	0
logged	0
pathbased	0
iop	0xffffffff80000b6f40
priv	0x0000000000000000

目录项 (DENTRY)
日志里没有这个文件系统的目录内容。

dev 20 **procfs · procfs** 来源 无 (伪设备)

按路径的伪文件系统。rootino 705468254 = FNV-1a("/") & 0x7fffffff (已用 Python 复算核对)。

磁盘 SUPERBLOCK (块 1)
日志里没有磁盘 superblock。

VFS SUPER_BLOCK (REGISTER_SUPER)	
dev	20

type	procfs
rootino	705468254
readonly	1
logged	0
pathbased	1
iop	0xffffffff80000b6bc8
priv	0xffffffff80000b6b70

目录项 (DENTRY)
日志里没有这个文件系统的目录内容。

dev 21 **fat32 /boot · fat32** 来源 mmcblk0p1

按路径的 FAT32。rootino 同样是 "/" 的哈希；priv 指向 fat32_pops。

磁盘 SUPERBLOCK (块 1)
日志里没有磁盘 superblock。

VFS SUPER_BLOCK (REGISTER_SUPER)	
dev	21
type	fat32
rootino	705468254
readonly	0
logged	0
pathbased	1
iop	0xffffffff80000b6bc8
priv	0xffffffff80000b6c28

目录项 (DENTRY)
日志里没有这个文件系统的目录内容。

dev 22 **netfs · netfs** 来源 192.168.0.195:5640

按路径的只读 netfs；priv 指向 netfs_pops。

磁盘 SUPERBLOCK (块 1)
日志里没有磁盘 superblock。

VFS SUPER_BLOCK (REGISTER_SUPER)	
dev	22
type	netfs
rootino	705468254
readonly	1
logged	0

logged	0
pathbased	1
iop	0xffffffff80000b6bc8
priv	0xffffffff80000b6c80

目录项（DENTRY）
日志里没有这个文件系统的目录内容。

路径查找图：inode → 目录项 → 数据块

选一条路径。每一行左边是当前目录的 inode，右边是它的目录数据块；箭头从 `addr[s[i]]` 指到磁盘块，再从命中的目录项指到下一级 inode。最后一行是目标文件的 inode 与它的数据块，样式与 ext2 inode 的块映射图相同。

 命中的目录项 本次新建的目录项 查找失败 无意义的残留值 `follow_mount` 穿过挂载点

PREPARE_NAMESPACE · ROOTFS

/dev

父 7:1 · 新建

1 级

/dev/console

父 7:2 · 新建

2 级

/root

父 7:1 · 新建

1 级

/dev/root

父 7:2 · 新建

2 级

/dev/root

→ 7:5

2 级

/root

→ 7:4

1 级

打开、执行与挂载

/root

→ 1:1

1 级

dev

→ 1:54 · ×2

1 级

/

→ 7:1

0 级

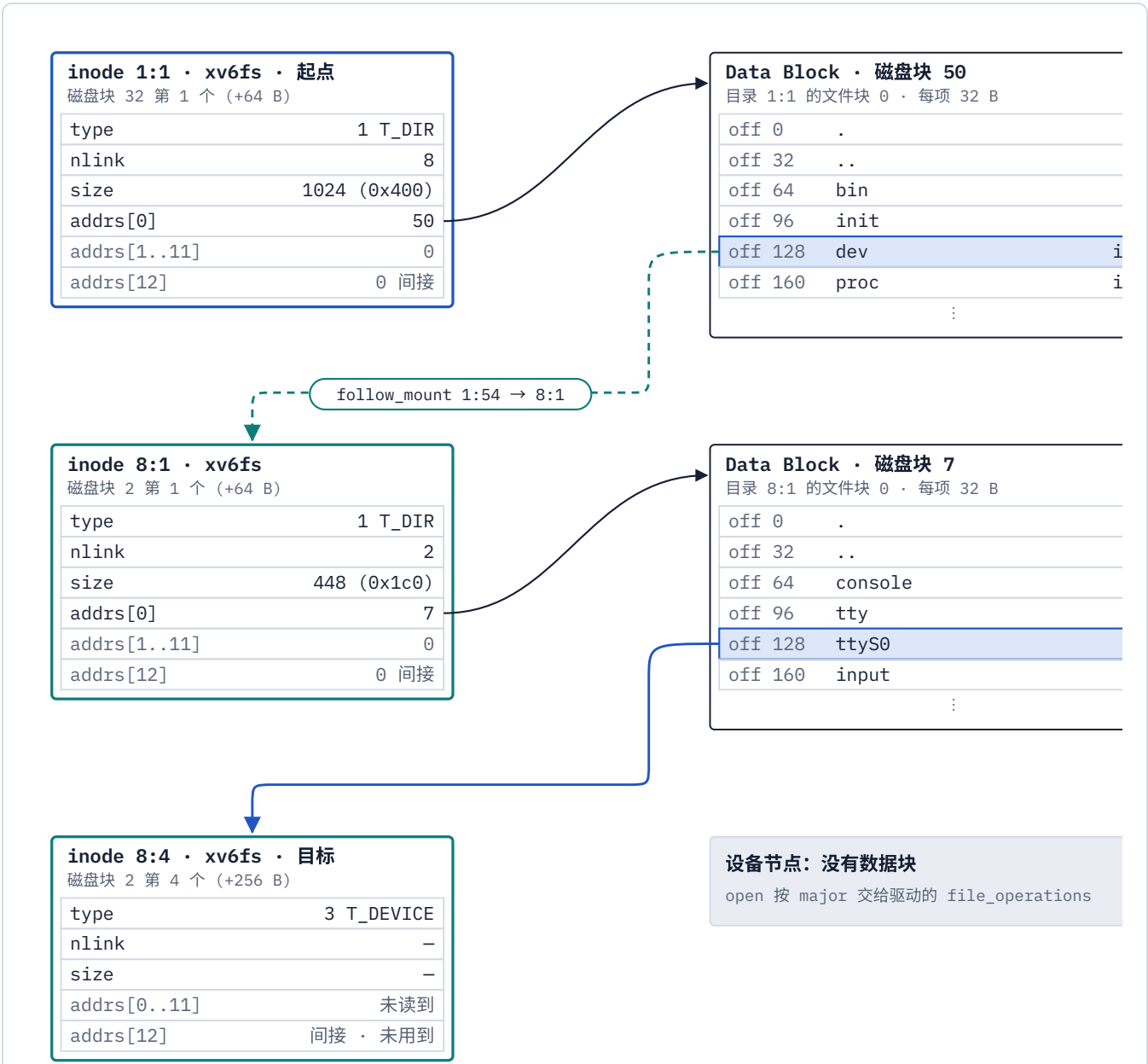
/init	1 级
→ 1:7	
/dev/ttyS0	2 级
→ 8:4	
/etc/hostname	2 级
→ 1:64	

日志第 2538 行

/dev/ttyS0

namei: 找到目标本身

- 1. rootdir() 返回 fs_root = 1:1
- 2. 在目录 1:1 里找 'dev': off 128 命中, ino 54; 1:54 是挂载点, follow_mount 换成 dev 8 的根 8:1
- 3. 在目录 8:1 里找 'ttyS0': off 128 命中, ino 4
- 4. 结果 inode 8:4, dinode 在磁盘块 2 第 4 个



每一级都是 `namex()` 调用 `iop->lookup()`：先 `ilock` 读目录 `inode`，再用 `bmap` 把 `addrs[]` 换成磁盘块，每次 `readi` 读 32 字节的目录项。

从日志看到的问题

三个 `xv6fs` 实例的布局与日志一致

`rootfs` (32 块)、`devtmpfs` (64 块)、根文件系统 (32768 块) 按 `superblock` 算出的数据区首块分别是 5、7、50，正好是三者根目录 `inode` 的 `addrs[0]`。`dinode` 位置也对得上： $\text{ino } 25 = 32 + 25/16 \rightarrow \text{块 } 33 \text{ 第 } 9 \text{ 个}$ 。

目录查找是线性扫描，每项一次 `readi`

每个 `dirent` 32 字节 (`DIRSIZ = 30`)，`dirlookup` 逐项 `readi`→`bmap`→`bread`。`/bin/wifi` 要读完 `/bin` 的前 25 项 (`off 0-768`) 才命中；整份日志里约 770 次 `readi` 调用，绝大多数是这种 32 字节的目录项读取，靠块缓存命中，没有真正读盘。

`/init` 和 `/bin/init` 是同一个 `inode`

根目录 `off 96` 和 `/bin off 192` 两个目录项都指向 `ino 7`，是硬链接。`exec("/init")` 读到的 5 个数据块是 130-134。

`devtmpfs` 的根 `inode` 被反复从磁盘读回

`devtmpfs_init()` 每建一个节点，根目录 8:1 的引用都降到 0，`itable` 槽被当作空槽重新初始化，下次又 `cache miss` 再读 `dinode`。日志里 8:1 被重新装载了 12 次。`xv6` 的 `inode` 表不保留引用为 0 的 `inode`，这是原版行为。

`ext2` 根 `inode` 打印出的 `addrs[0]=1660` 是残留值

`itable[6]` 之前装的是 `/mnt (1:57)`，它的目录块正是 1660。`e2fill()` 只填 `type/nlink/size`，不清 `addrs[]`，所以跟踪输出里的 `addrs` 对 `ext2` 没有意义。建议在 `e2_read_inode()` 里清零 `addrs`，或者跟踪代码对非 `xv6fs` 不打印 `addrs`。

按路径的文件系统共用一个根 `inode` 号

`procfs`、`fat32`、`netfs` 的 `rootino` 都是 705468254，即 `/` 的 31 位 FNV 哈希。它们靠设备号 20/21/22 区分，`iget_path` 还会比较 `path` 字符串。

有一次失败的查找

第 2418 行查找 `/console`：根文件系统根目录里没有这个名字，`xv6_lookup` 返回 `NULL`，调用者随后转去打开 `/dev/ttyS0`。

日志在 `/bin/login` 的 `exec` 中途结束

最后一行读到 `login` 的数据块 3（磁盘块 202），后面的段没有记录。`ext2` 目录块、`FAT32` 和 `procfs` 的读取也没有跟踪点。所以 `/mnt/ext2/video` 的 `ino 12` 是从下一次 `iget` 推断的。